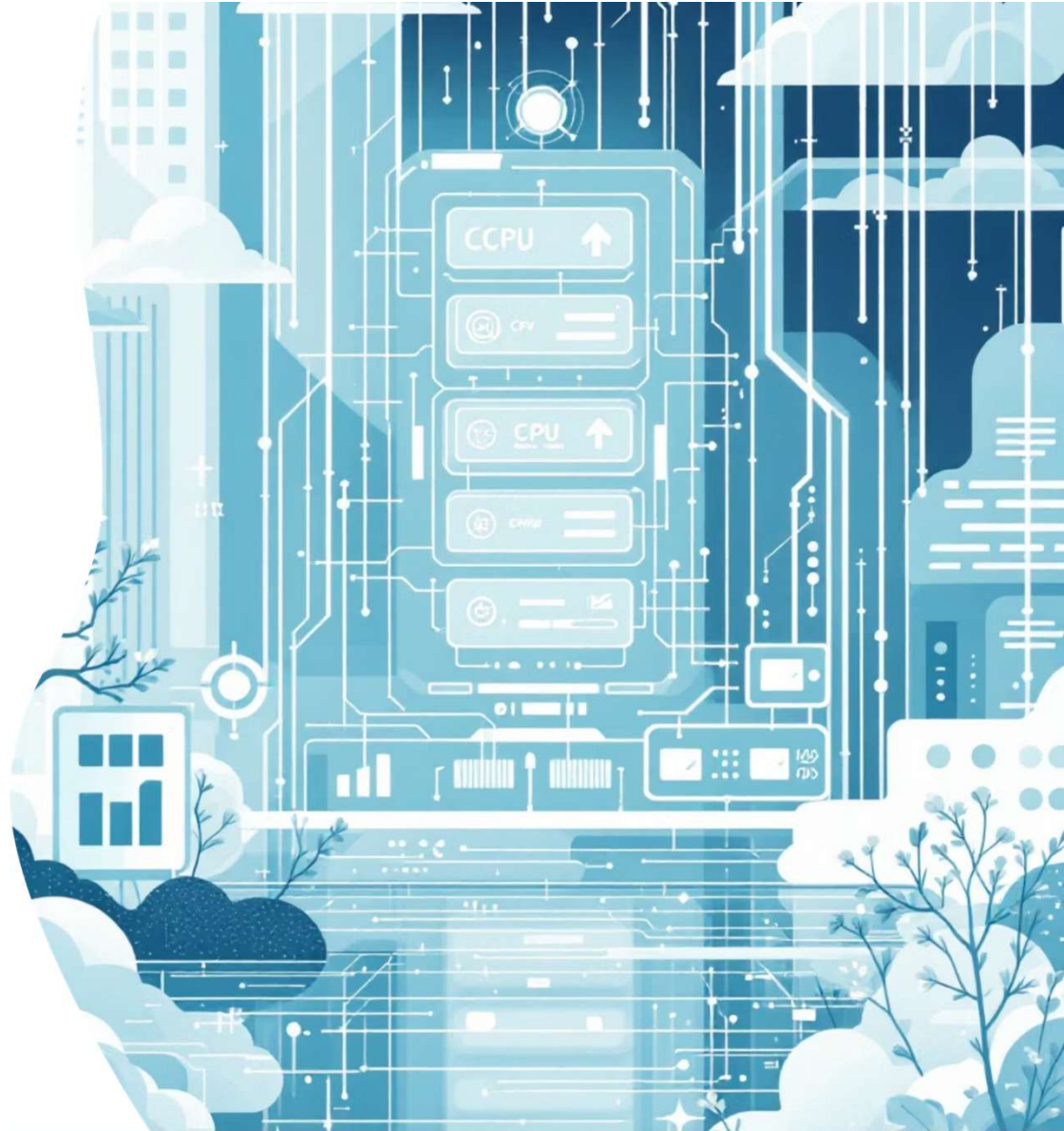


Adaptive Resource Allocation Simulator in Multiprogramming Systems

A visual demonstration of dynamic resource allocation



Understanding the Foundation

Introduction to Multiprogramming & Resource Allocation

Multiprogramming

- Allows a single CPU to execute multiple programs concurrently
- Creates illusion of parallelism
- Improves CPU utilisation

Why Adaptive resource allocation matters:

The core problem in multiprogramming systems is balancing resource demand with availability to maintain optimal performance. An adaptive approach ensures that resources are always distributed to where they are needed most, improving overall system responsiveness.

Project Goal

To visually demonstrate dynamic resource allocation strategies within a multiprogramming system.

The Challenge: Managing Limited System Resources

Processes Compete for Resources

- Multiple processes request CPU & memory
- Requires intelligent management

Finite Capacity

- CPU: 100%
- Memory: 2048 MB

Need for Adaptive Scheduling

- Dynamically adjusts allocations
- Ensures efficient performance

System Architecture Overview

Our simulator is divided into three modules:

Module 1: Process Management

What this module covers:

- Creating processes with CPU and memory requests
- Assigning unique PIDs
- Managing process states (Ready, Running, Waiting, Completed)
- Deleting processes
- Resetting the simulator

Why it's important: Foundation of the simulator — no processes means no allocation.

Module 2: Resource Allocation & Scheduling

What this module covers:

- Memory availability checks
- CPU percentage distribution
- Moving processes between Ready/Running/Waiting
- Updating progress
- Ensuring realistic execution

Why it's important:

Makes the simulator "smart" by preventing resource over-allocation.



Module 3: User Interface & Visualization

1

Interface Design

Designing the Tkinter interface

2

Usage Bars

CPU and memory usage bars

3

Process Table

Process table

4

Progress Tracking

Progress bars

5

Controls

Add/Delete/Reset buttons

6

Live Updates

Real-time updates

Why it's important: Makes the simulation interactive and understandable.

Understanding Process States



Ready

Waiting for CPU/memory



Running

Actively executing



Waiting

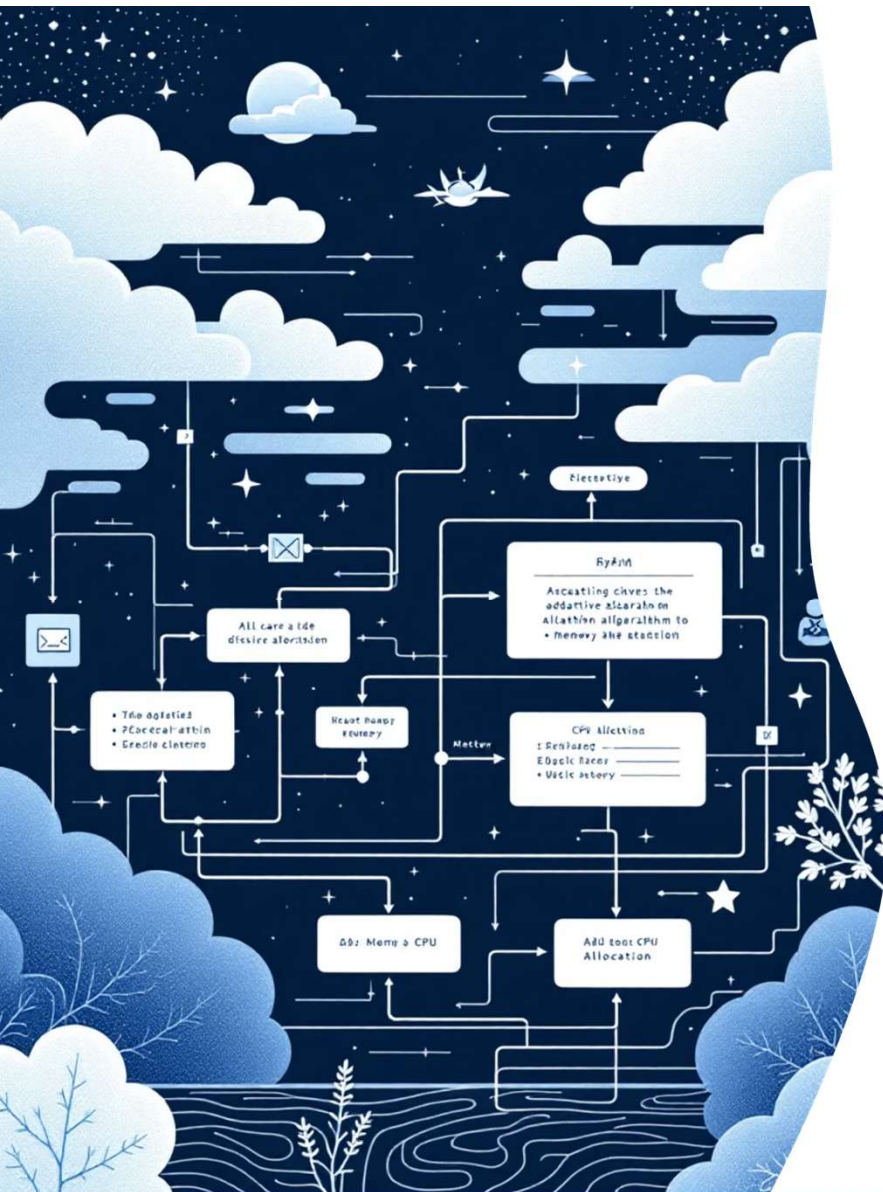
Paused due to unavailable resources



Completed

Finished execution and released resources

State transitions occur based on CPU and memory availability.



Adaptive Allocation Algorithm

Memory Allocation Rule:

- Process runs only if full memory is available

CPU Allocation Rule:

- Scheduler uses a policy (Round Robin / SJF)
- CPU time dynamically shared

Progress Rule:

- Progress updates continuously
- Upon completion, resources return to the system

What We Learned

Key Insights & Observations



CPU Control

CPU usage never exceeds 100%



Memory Limits

Memory always stays within 2048 MB



Dynamic Competition

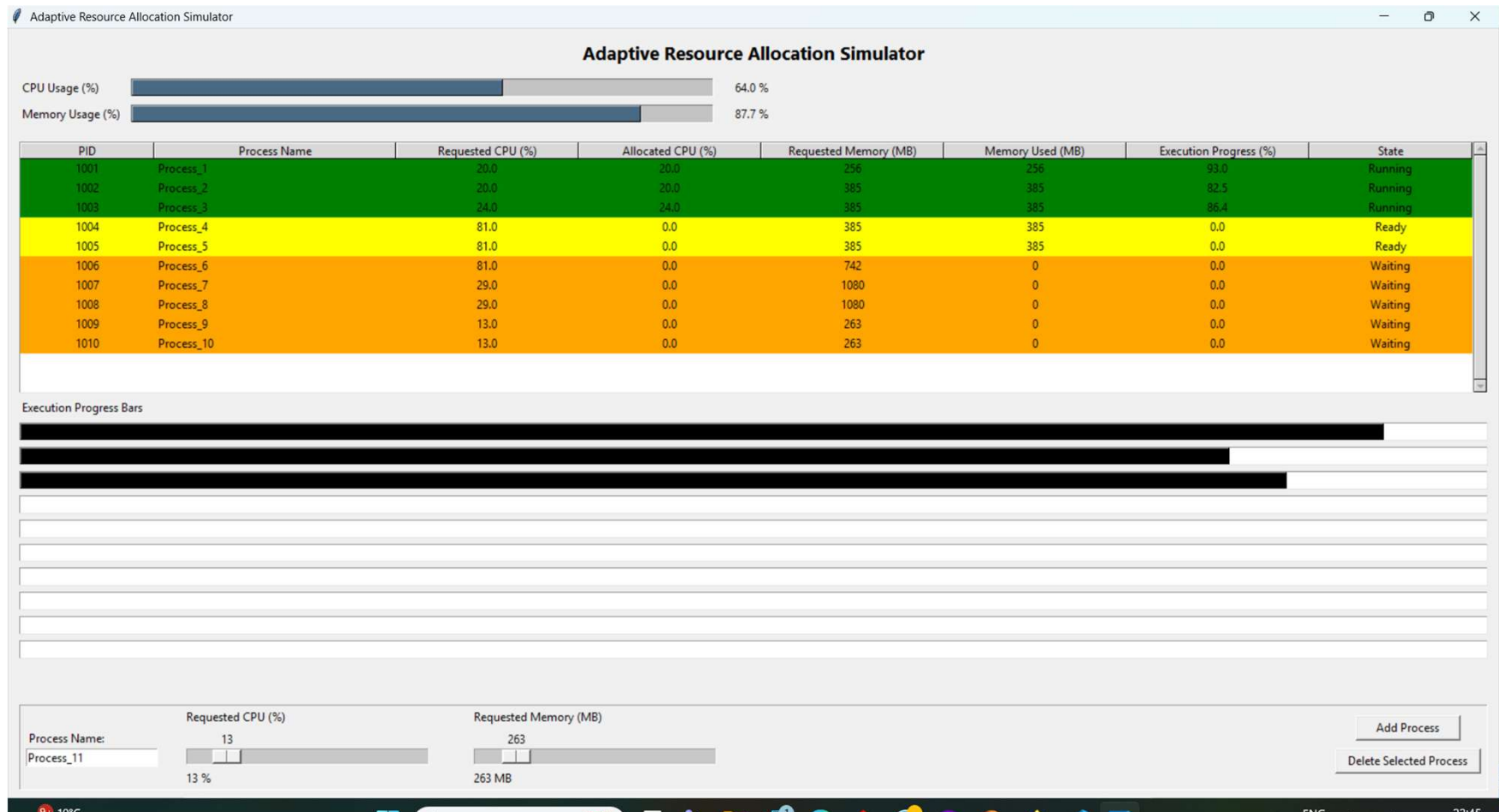
Processes dynamically compete for resources

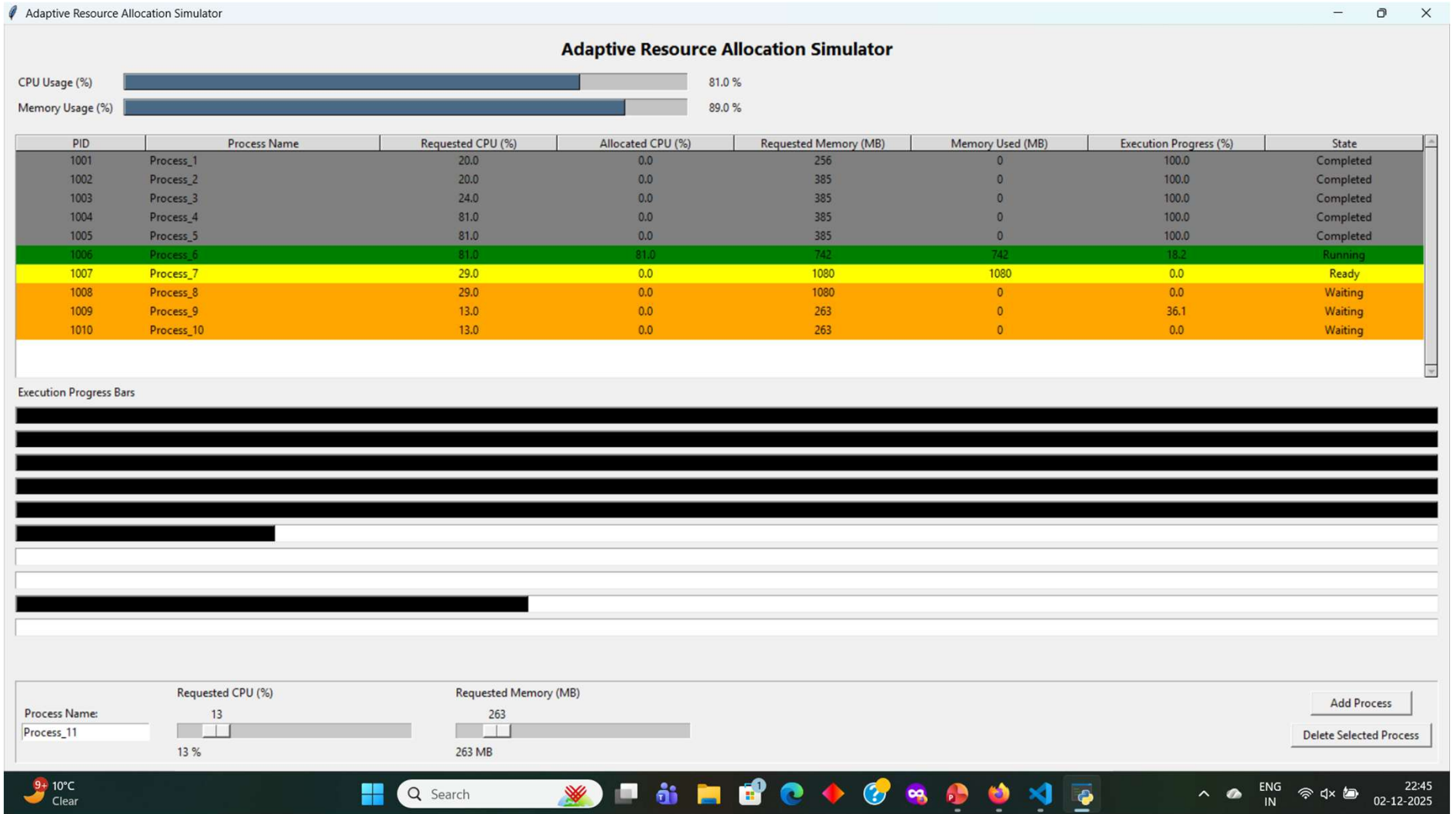


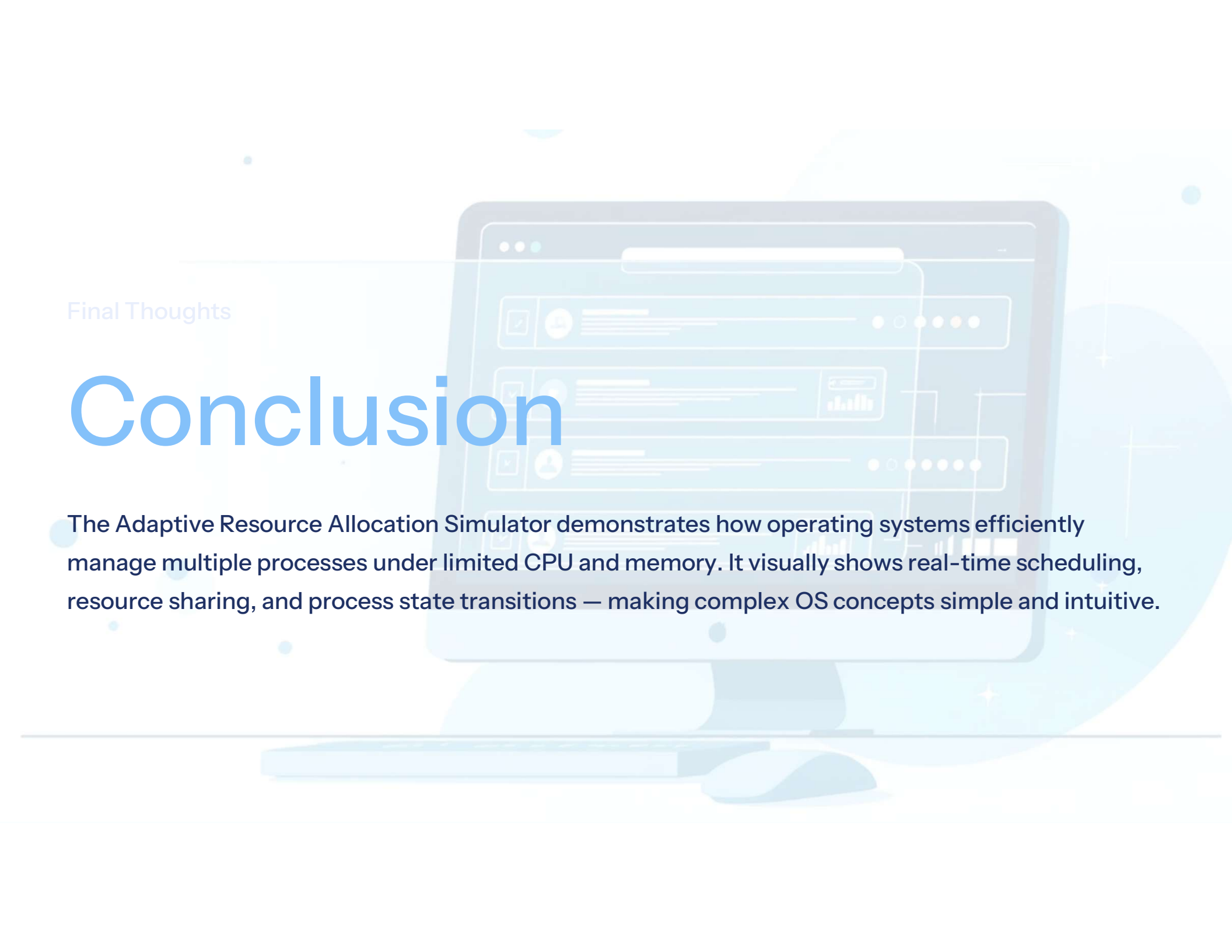
Realistic Behavior

Simulator accurately reflects OS behaviour

Screen Shots:







Final Thoughts

Conclusion

The Adaptive Resource Allocation Simulator demonstrates how operating systems efficiently manage multiple processes under limited CPU and memory. It visually shows real-time scheduling, resource sharing, and process state transitions — making complex OS concepts simple and intuitive.